

软件部软件开发规范 V4.1

[[内部资料： 密级]]

版本历史

版本/状态	作者	参与者	开始日期	结束日期	备注
V1.0	查治国		2007-9-10	2007-9-11	初始版本
V1.1	宋波		2007-12-14	2007-12-17	增加一些编码规范
V2.0	查治国		2008-1-11	2008-1-17	
V3.0			2011-6-13		2011-6-13 为了适用新的需求，重新改写此规范
V3.1	李琴		2012-4-11	2012-4-16	增加 Android 平台开发规范。
	李琴		2012-11-5	2012-11-5	增加签名、版本号章节。
V4.0	李琴		2013-10-17	2013-10-18	修订 Android 平台开发规范。
	李琴		2014.06.18	2014.06.18	修订 Android 平台开发规范。
V4.1	李琴		2014.12.19	2014.12.19	修订 Android 平台开发规范中 5.2、5.3.1 和 5.3.2。

目 录

软件部软件开发规范 V4.1.....	1
版 本 历 史.....	2
目 录.....	3
一、概述.....	4
二、功能需求分析.....	4
三、概要设计.....	4
四、FeaturePhone 平台开发规范.....	4
4. 1 工程目录.....	4
4. 2 编译开关选项.....	6
4. 3 代码编写.....	6
4.3.1 统一的修改标记符号.....	7
4.3.2 命名规范.....	7
4.3.4 文件规范.....	7
4.3.5 函数规范.....	8
4.3.6 编译规范.....	8
五、Android 平台开发规范.....	8
5.1 工程目录.....	8
5.2 应用的独立性.....	8
5.2.1 基于 ABS 架构的条件定义和配置说明.....	9
5.2.2 基于 AOSP 架构的条件定义和配置说明.....	10
5.3 代码编写.....	10
5.3.1 统一的修改标记符号.....	10
5.3.2 命名规范.....	11
5.3.3 文件规范.....	12
5.3.4 其他编码规范.....	15
5.3.5 签名.....	18
5.3.6 版本号.....	20
六、软件测试.....	22
七、代码审查.....	22
八、资料整理.....	22
九、参考资料.....	22

一、概述

为了使软件开发过程有章可循，保证软件质量，加强软件开发管理。通过增强代码的可重用性和易读性等方面来提高软件开发质量，降低开发周期，使软件便于维护和移植，开发人员间便于交流和协作。通过制定此规范，并将此规范将作为软件所系统软件和应用软件验收的依据来加强软件开发管理。

整个软件的开发须遵守下面一些基本的原则：软件工程化；模块化；能简单不复杂；强调团队协作；强调创新和特色。

软件项目开发周期分为以下几个步骤：功能需求分析；概要设计；代码编写；软件测试；资料整理发布。

二、功能需求分析

对该软件项目进行详细的需求分析，如果有 SPEC，需要仔细的阅读SPEC。了解清楚有那些功能需求，最终要做什么样子。

在项目需求分析阶段，要确认完成此功能开发，需要的资源，包含内部资源，外部资源等条件。

三、概要设计

概要设计编制的目的是说明对程序系统的设计考虑，包括程序系统的基本处。流程、程序系统的组织结构、模块划分、功能分配、接口设计、运行设计、数据结构设计和出错处理设计等。

四、FeaturePhone 平台开发规范

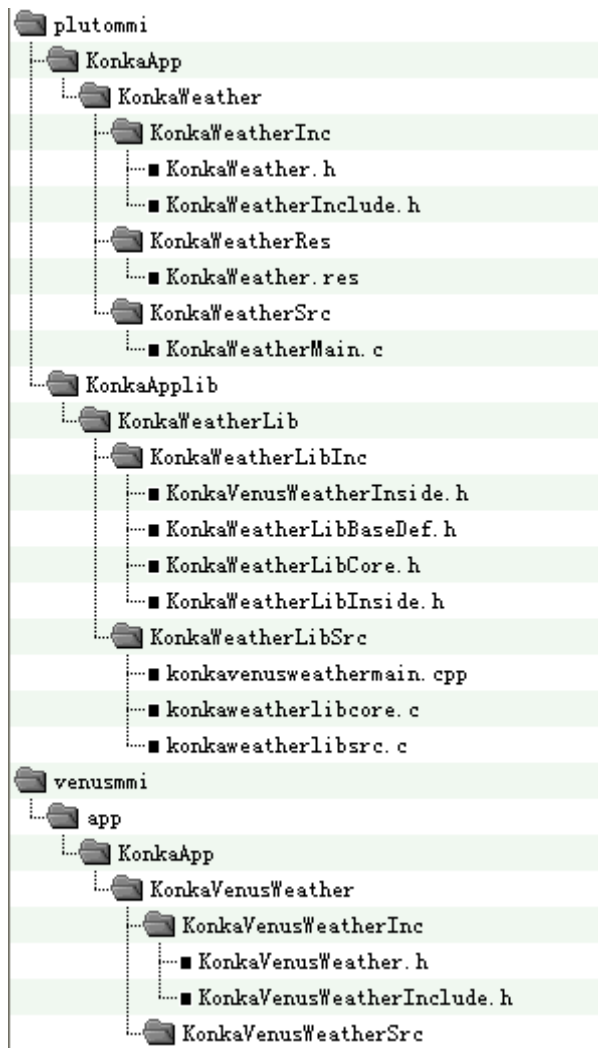
功能软件是独立于项目组开发的，而且要应用于不同MTK 平台和不同的项目，这就要求我们开发的应用程序移植性要好，可以快速，平滑的在不同平台，不同项目之间移植。为了保证这个功能软件在不同的项目中同步更新，而不会出现某个问题，一个项目改，其他项目或者后续项目都没修改等问题，我们需要把这个软件代码封装成库，提供给项目，这些完成后专门人来负责修改，同步更新。

在项目的概要设计中，要明确这个代码是否要封装成库，定义好整个软件功能工程；要定义好整个软件功能的控制宏。针对 MTK feature phone 平台的特性，制定开发规范如下。

4. 1 工程目录

方便、合理的软件系统目录，可提高工作效率。目录构造的原则是方便有关源程序的存储、查询、编译、链接等工作。根据软件功能的分析，定义好此功能的工程目录和文件命名，

文件的命名规则是能表达此文件的主要意思的英文名，一般一个源文件匹配一个头文件。
下面是《天气预报》的工程目录定义



KonkaApp\KonkaWeather: 该目录包含的此功能移植到某项目有关的文件和资源定义，也就是一些项目可以更改的内容

KonkaWeather.h: 此功能的头文件，主要申明一些给外部项目使用的函数，结构等

KonkaWeatherInclude.h: 此功能要使用的系统所有头文件的集合

KonkaWeather.res: 此功能的资源定义

KonkaWeatherMain.c: 为了匹配此功能，项目需要修改的一些函数或者变量

KonkaApplib\KonkaWeatherLib: 此功能的代码（可以编译成库）

KonkaWeatherLibBaseDef.h: 此功能使用的一些资源类ID 的定义，从0 开始，顺序必须与资源定义想对应，使用的时候通过加一个偏移变量来获取真实的 ID。一般使用的时候有：MENU、STRING、IMAGE、NVRAM、SCREEN、TIMER、CATEGORY 等。

KonkaWeatherLibInside.h: 功能内部使用的结构、变量和函数说明

KonkaVenusWeatherInside.h: 功能Venus 部分使用的结构、变量和函数说明

KonkaWeatherLibCore.h: 此功能核心代码使用的结构、变量和函数说明

konkawweatherlibsrc.c: 此功能的UI 代码，主要是界面显示等部分

konkavenusweathermain.cpp: 此功能的Venus 代码，主要是类的实现部分

konkawweatherlibcore.c: 此功能的核心代码，主要是算法的处理等API

KonkaApp\KonkaVenusWeather: 该目录包含的此功能Venus APP SHELII 移植到某项目有关的文件和资源定义, 也就是一些项目可以更改的内容

KonkaVenusWeather.h: 类的定义

KonkaVenusWeatherInclude.h: 此功能要使用的系统所有头文件的集合

4. 2 编译开关选项

每个项目对功能的需求不一样, 为了使项目可以非常容易的打开、关闭某一个应用功能。要实现此要求, 我们必须使用编译开关选项来达到此要求, 通过编译开关选项的开启和关闭来实现应用功能的开启、关闭。在使用要统一编译开关选项来控制某个功能时候, 要看这个功能属于哪个大的类别, 要大类的编译开关也能控制这个功能, 而且在命名规则上还需要统一编译开关选项。

我们对功能有分类, 一个大类下面有许多功能, 这样要求每个功能宏控制的时候, 需要包含大类的宏和本身的宏。新增加功能。

使用编译开关选项控制的功能, 一定要检查编译开关选项关闭后是否对整个项目造成影响。下面是其他类别的举例:

```
KONKA_OTHER_SUPPORT = TRUE #TRUE or FALSE
ifeq ($(strip $(KONKA_OTHER_SUPPORT)),TRUE)
#其他总控制宏
CUSTOM_OPTION += __KONKA_OTHER__
#设置菜单双行列表显示功能
CUSTOM_OPTION += __KONKA_MMI_DB_LIST_FOR_SETTING__
CUSTOM_OPTION += __KONKA_MMI_GENE_MENU__
#EMAIL
CUSTOM_OPTION += __KONKA_MMI_EMAIL__
#电量查看
CUSTOM_OPTION += __KONKA_MMI_ELECTRIC_POWER__
#电灯泡
CUSTOM_OPTION += __KONKA_MMI_BULB__
#天气预报
CUSTOM_OPTION += __KONKA_MMI_WEATHER__
CUSTOM_OPTION += __KONKA_VUI_WEATHER__ #VENUS 的MMI
endif
在天气预报中使用例子:
#if defined(__KONKA_OTHER__) && defined(__KONKA_MMI_WEATHER__)
```

4. 3 代码编写

这部分主要是对编写的代码做出一些规范性的要求, 主要有以下部分: 有些不易理解的变量或函数应作注释, 难懂的代码要有注解, 在文件的开始处有该文件的用途描述。一定要保持注释的一致性。

代码组织要清晰, {, }, (,), if, else, do, while, for, case 等要对应整齐, 少用空格, 缩进全部用Tab 键。变量的定义要集中, 函数间要有空行分开, 一个程序中的空行数目最好占8%-16%。多态函数和功能相近的函数集中放在一起。

代码应该简洁、清楚并讲述了所发生的一切, 我们的目标应该是写出最清晰的代码, 而

不是最巧妙的代码。

在代码中尽量不用具体的大小数值，定义成宏，便于以后维护。

4.3.1 统一的修改标记符号

修改的内容使用统一的修改标记符号，格式如下：

修改开始标记：

```
/*konka—XXX(修改者名字)—XXXX(功能)—XXXX(修改日期 20070910) —start */
```

修改结束标记：

```
/*konka—XXX(修改者名字)—XXXX(功能)—XXXX(修改日期 20070910) —end*/
```

对于独立使用宏控制的，可以不加标记；

对于本身功能易用性的修改，使用的系统本身的宏，则需要加修改标记；

功能已经正式发布，后续 bug 的修改，要加修改标记。

4.3.2 命名规范

命名应尽量使用匈牙利命名法，变量名或函数名中使用大写字符来区分各个部分，以便于记忆和阅读。如bPatchMinute, DeleteDirInfo()。全局(包括类中的)变量用长名字，局部变量用短名字。

类成员变量前一般应加上m_，全局变量加上g_，仅与本模块有关的变量加上l_，紧接着是变量的类型。

对宏、全局变量做必要的说明；结构体，联合体，枚举的成员做必要的说明，说明这些的用途，全局变量在定义的时候，必须初始化。

更具体的信息，可以参考《匈牙利命名法》

4.3.4 文件规范

原则上一个源文件 (*.c/*.cpp) 必须至少匹配一个头文件 (*.h)，每个源文件和头文件都必须有一段格式统一的说明信息。如下所示：

```
/******
```

```
* 版权所有 (C) 2010，深圳市康佳集团股份有限公司。
```

```
系统组软件开发规范
```

```
第 8 页
```

```
*
```

```
* 文件名称: KonkaWeather.h
```

```
* 文件标识:
```

```
* 内容摘要: 天气预报的头文件, 说明外部可以调用的函数
```

```
* 其它说明:
```

```
* 当前版本: 1.0
```

```
* 作者:
```

```
* 完成日期:
```

```
* 修改记录:
```

- * 修改日期:
- * 版本号:
- * 修改人:
- * 修改内容:

*****/

修改记录依照时间顺序从上到下记录

4.3.5 函数规范

函数要写的简单，易懂，组织结构要清晰，简洁。一个函数就表示一个功能，不要把一个函数写的太复杂，需要完成很多功能。在函数里，不易理解的变量或函数应作注释，比较复杂或者特殊的调用也要作注释，帮助别人很好的理解代码。

在代码中尽量不用具体的大小数值，定义成宏，便于以后维护。

删掉从来没有用过的函数或变量，大篇幅注释掉的代码行也应删除，以免使程序混乱难读。

函数应有注释说明。内容包括：功能、入口/出口参数，必要时还可有备注或补充说明，对于函数中的主要功能代码，也应有注释说明。如下例子：

/******

- * 函数名称: KonkaWeatherCreateParseCityData
- * 功能描述: 创建城市解析信息
- * 输入参数: eDataType 数据源类型
- * 输出参数: lpTotal
- * 返回值: 城市信息列表
- * 备注: 无

*****/

4.3.6 编译规范

打开编译器的所有告警开关对程序进行编译，并消除警告错误。同时验证编译选项开关，能否编译通过。

五、Android 平台开发规范

针对 MTK Android 平台应用开发特性，参考 java 编码特点，制定规范如下。

5.1 工程目录

软件部开发的新应用统一放在 alps/packages/apps 目录下。

5.2 应用的独立性

每个独立应用都使用 feature option 条件进行控制，使得项目可根据需要选择开启或关

闭该应用。

feature option 条件命名规范遵循如下规则：

KONKA_XXXX_XXXX

例如：

语音助手功能的 feature option: KONKA_SPEECH_CLIENT

康佳信息的 feature option: KONKA_MESSAGE

5.2.1 基于 ABS 架构的条件定义和配置说明

在 Android 版本升级到 5.0 之前，MTK 提供的 codebase 都是基于 ABS 架构的。在此架构的基础上，使用统一的配置文件，可以定义我们需要的条件，以及配置该条件的属性。

- 1、feature option 条件定义在 alps\mediatek\build\tools\javaoption.pm 文件中。
- 2、feature option 条件的开关由 alps\mediatek\config 目录下公共（common）或项目（konkaXX_XX_XXX）中的文件 ProjectConfig.mk 进行设置。
- 3、在 javaoption.pm 和 ProjectConfig.mk 中进行条件定义和条件配置时，基本上遵循**新增内容添加至文件末尾**的原则；同时，为了 git 仓库管理的方便，新增的条件定义和配置，需遵循一定格式，例如：

1) javaoption.pm文件中最后部分：

```
# konka -FindSilentPhone-xietian duo -20140919-start
KONKA_FIND_SILENT_PHONE
# konka -FindSilentPhone-xietian duo -20140919-end
#
#
#
#konka -EngineerMode Battery log-chenliping -20140917-20141116-start
KONKA_BATTERYLOG_ADD_CHARGING
#konka -EngineerMode Battery log-chenliping -20140917-20141116-end
#
#
#
#
#
#
```

2) ProjectConfig.mk文件中最后部分

```
# konka -CameraVolumeResponse-xiaxiaoling -20140919-start
KONKA_CAMERA_VOLUME_RESPONSE = yes
# konka -CameraVolumeResponse-xiaxiaoling -20140919-end
#
#
#
# konka -FindSilentPhone-xietian duo -20140919-start
KONKA_FIND_SILENT_PHONE = yes
# konka -FindSilentPhone-xietian duo -20140919-end
#
```

```
#
#
#
#
```

说明：上述格式的规则是，文件末尾以6个#号结束，每两项内容之间，使用3个#号分隔。这样，在git仓库中，这两个文件在本地的版本与服务器版本产生merge时，会自动合并而不产生冲突。

该规则同理可应用于其他常用文件例如alps\build\target\product\common.mk，前提是对该文件的修改也遵循**新增内容添加至文件末尾**的原则。

5.2.2 基于 AOSP 架构的条件定义和配置说明

在 Android 版本升级到 5.0 之后，MTK 提供的 codebase 开始遵循 google 的标准 AOSP 架构。因此，原来的 feature option 条件定义和配置发生了一定变化。

1、首先，feature option 条件在 alps\device\mediatek\common\device.mk 中作为条件，来决定其对应的 System Property 属性值，例如：

```
ifeq ($(strip $(MTK_FOOBAR_SUPPORT)), yes)
    PRODUCT_PROPERTY_OVERRIDES += \
        ro.mtk_footbar_support=1
endif
```

如上图示中，最终在源代码中被引用的是 System Property 即图中的 ro.mtk_footbar_support。

2、其次，feature option 条件在 alps\device\konka\[project_name]\ProjectConfig.mk 中进行开关的配置，例如：

```
MTK_FOOBAR_SUPPORT = yes
```

3、针对上面 1、2 两项所述文件的修改，同样为了 git 仓库管理的方便，我们希望大家在使用时遵循如下规则：

- 1) 新增内容添加至文件末尾；
- 2) 文件末尾以 6 个#号结束，每两项内容之间，使用 3 个#号分隔。

5.3 代码编写

5.3.1 统一的修改标记符号

对工程原生代码进行修改时，为了方便查阅和甄别内容，修改的内容在开始前和结束后都需使用统一格式的标记符号做注释，格式规定如下：

修改开始标记：

```
/*konka- XXXX(功能名称或修改描述)- XXX(修改者名字)-YYYYMMDD(修改日期 20070910) -start */
```

修改结束标记:

```
/*konka- XXXX(功能名称或修改描述)- XXX(修改者名字)-YYYYMMDD (修改日期 20070910) -end*/
```

补充说明:

1、 功能已经正式发布, 后续 bug 的修改, 也要加修改标记。针对功能修改 bug 时, 新增的注释要求保证“XXXX(功能名称或修改描述)”部分与原有功能保持**功能名称统一**, 可在统一的功能名称之后再增加对修改细节的描述, 例如:

```
/*konka- KonkaDynamicCalendar- xiaxiaoling-20140610-start*/
```

```
/*konka- KonkaDynamicCalendar- xiaxiaoling-20140610-end*/
```

上述两行是注释中标识为 KonkaDynamicCalendar 的“动态日历”功能修改到原生代码或者其他新增功能模块代码时, 修改内容的开始和结束标记注释。

而后来由于新需求产生或者问题修正, 需要对“动态日历”功能的部分代码进行修改, 则负责修改的工程师把修改内容用如下注释来作为他所添加的修改内容开始结束的标记:

```
/*konka -KonkaDynamicCalendar--yanglehui 20140822-start*/
```

```
/*konka -KonkaDynamicCalendar--yanglehui 20140822-end*/
```

上述注释中, 遵循了**功能名称统一**的原则, 而没有对修改进行更细节的描述。如果添加描述的话, 则注释会是下面的样子:

```
/*konka -KonkaDynamicCalendar:fix bug xxxx--yanglehui 20140822-start*/
```

```
/*konka -KonkaDynamicCalendar:fix bug xxxx--yanglehui 20140822-end*/
```

2、 上述标记在不同文件中有不同展现形式, 请工程师灵活处理。例如, 对原生*.xml 文件增加修改时, 如下:

修改开始标记:

```
<!-- konka- XXXX(功能名称或修改描述)- XXX(修改者名字)-YYYYMMDD (修改日期 20070910) -start -->
```

修改结束标记:

```
<!-- konka- XXXX(功能名称或修改描述)- XXX(修改者名字)-YYYYMMDD (修改日期 20070910) -end -->
```

5.3.2 命名规范

命名规范使程序更易读, 从而更易于理解。它们也可以提供一些有关标识符功能的信息, 以助于理解代码, 例如, 不论它是一个常量, 包, 还是类。

标识符类型	命名规则	例子
包(Packages)	一个唯一包名的前缀总是全部小写的ASCII字母并且是一个顶级域名, 通常是com, edu, gov, mil, net, org, 或1981年ISO 3166标准所指定的标识国家的英文双字符代码。 而我们在开发过程中, 规定包名命名参照如下方式: com.android.xxxxx 注: 由于当前外销产品的业务模式, 客户不希望在产品中出现konka字样, 所以目前开发的功能包名中也请先规避konka。	com.android.helloworld
类(Classes)	命名规则: 类名是个一名词, 采用大小写混合的方式, 每个单词的首字母大写。尽量使你的类名简洁而富于描述。使用完整单词, 避免缩写词(除非该缩	<pre>class Raster; class ImageSprite;</pre>

	写词被更广泛使用，像URL，HTML)	
接口 (Interfaces)	命名规则：大小写规则与类名相似	interface RasterDelegate; interface Storing;
方法(Methods)	方法名是一个动词，采用大小写混合的方式，第一个单词的首字母小写，其后单词的首字母大写。	run(); runFast(); getBackground();
变量(Variables)	变量名，以及所有实例，包括类，类常量，均采用大小写混合的方式，第一个单词的首字母小写，其后单词的首字母大写。变量名不应以下划线或美元符号开头，尽管这在语法上是允许的。变量名应简短且富于描述。变量名的选用应该易于记忆，即，能够指出其用途。尽量避免单个字符的变量名，除非是一次性的临时变量。临时变量通常被取名为i, j, k, m和n，它们一般用于整型；c, d, e，它们一般用于字符型。 类成员变量一般以m开头。	char c; int i; float myWidth; String mAppDir = null;
实例变量 (Instance Variables)	大小写规则和变量名相似，除了前面需要一个下划线	int _employeeId; String _name; Customer _customer;
常量 (Constants)	类常量和ANSI常量的声明， 应该全部大写 ，单词间用下划线隔开。(尽量避免ANSI常量，容易引起错误)	static final int MIN_WIDTH = 4; static final int MAX_WIDTH = 999; static final int GET_THE_CPU = 1;

5.3.3 文件规范

1、注释

1) 所有的源文件都应该在开头有一个 C 语言风格的注释，其中列出版权声明、文件名、版本信息、日期和作者等等，文件头信息如下：

```
/******
```

```
* 版权所有 (C)2012, 深圳市康佳集团股份有限公司。
```

```
*
```

```
* 文件名称:
```

```
* 内容摘要:
```

```
* 当前版本:
```

```
* 作 者:
```

```
* 完成日期:
```

```
* 修改记录:
```

```
* 修改日期:
```

* 版本号:
* 修改人:
* 修改内容:

...

* 修改记录:
* 修改日期:
* 版本号:
* 修改人:
* 修改内容:

*****/

修改记录依照时间顺序从上到下记录

2) 类名前的注释规定使用“文档注释”，参考格式如下（其中@ author、@ version 等文档标记参考“[5.3.4 其他编码规范](#)”中的“文档注释”）:

```
/**
 * 类描述 :
 * @author 作者
 * @version 版本
 */
```

3) 方法名前的的注释规定使用“文档注释”，参考格式如下（其中@param、@see 等文档标记参考“[5.3.4 其他编码规范](#)”中的“文档注释”）:

```
/**
 * 方法描述:
 * @param 参数名 说明
 * @return 返回类型 说明
 * @see 类名/完整类名/完整类名#方法名
 */
```

“文档注释”请参考“[5.3.4 其他编码规范](#)”章节。

2、包和引入语句(Package and Import Statements)

在多数Java源文件中，第一个非注释行是包语句。在它之后可以跟引入语句。例如：

```
package android.test;
import android.content.Intent;
```

3、类和接口声明(Class and Interface Declarations)

下面描述了类和接口声明的各个部分以及它们出现的先后次序。

- 1) 类/接口文档注释 (/**.....*/)
- 2) 类或接口的声明
- 3) 类/接口的实现注释

如果有必要的话，该注释应包含任何有关整个类或接口的信息，而这些信息又不适合作为类/接口文档注释。

4)、类的(静态)变量

首先是类的公共变量，随后是保护变量，再后是包一级别的变量(没有访问修饰符，access modifier)，最后是私有变量。

5) 实例变量

首先是公共级别的，随后是保护级别的，再后是包一级别的(没有访问修饰符)，最后是私有级别的。

6) 构造器

7) 方法

这些方法应该按功能，而非作用域或访问权限，分组。例如，一个私有的类方法可以置于两个公有的实例方法之间。其目的是为了更便于阅读和理解代码。

4、样例

```
/******
```

```
* 版权所有 (C)2012， 深圳市康佳集团股份有限公司。
```

```
*
```

```
* 文件名称: HelloWorld. java
```

```
* 内容摘要: This is an android demo which shows “hello world”.
```

```
* 当前版本: V1.0
```

```
* 作 者: Konka
```

```
* 完成日期: 2012-4-12
```

```
* 修改记录:
```

```
* 修改日期:
```

```
* 版本号:
```

```
* 修改人:
```

```
* 修改内容:
```

```
*****/
```

```
package android.HelloWorld;
```

```
import android.app.Activity;
```

```
/**
```

```
* 此处对类进行描述 *
```

```
*/
```

```
public class HelloWorld extends Activity {
```

```
/*类/接口的实现注释*/
```

```
/**公共静态变量的文档注释 */
```

```
public static int classVar1;
```

```
/**
```

```
* 私有静态变量的文档注释（多行情况）
```

```
*/
```

```
private static Object classVar2;
```

```
/** 公共级别实例变量的文档注释 */
```

```
public Object instanceVar1;
```

```
/** 保护级别实例变量的文档注释 */
protected int instanceVar2;
/** 私有级别实例变量的文档注释 */
private Object[] instanceVar3;
/**
 * ... 构造器HelloWorld的文档注释...
 */
public HelloWorld () {
// ... 构造器的补充描述注释...
}
/**
 * ... 方法doSomething的文档注释...
 */
public void doSomething() {
// ... 方法的补充描述注释...
}
}
```

补充说明：上述例子中的“实现注释”和“文档注释”请参考“[5.3.4 其他编码规范](#)”章节。

5.3.4 其他编码规范

1、注释

Java 程序有两类注释：实现注释(implementation comments)和文档注释(document comments)。实现注释是那些在 C++中见过的，使用/*...*/和//界定的注释。文档注释(被称为“document comments”)是 Java 独有的，并由/**...*/界定。文档注释可以通过 javadoc 工具转换成 HTML 文件。

1) 实现注释(Implementation Comment)

块注释

块注释通常用于提供对文件，方法，数据结构和算法的描述。块注释被置于每个文件的开始处以及每个方法之前。它们也可以被用于其他地方，比如方法内部。在功能和方法内部的块注释应该和它们所描述的代码具有一样的缩进格式。

块注释之首应该有一个空行，用于把块注释和代码分割开来，比如：

```
/*
 * Here is a block comment.
 */
```

块注释可以以/*-开头，这样indent(1)就可以将之识别为一个代码块的开始，而不会重排它。

```
/*-
 * Here is a block comment with some very special
 * formatting that I want indent(1) to ignore.
```

```
*
* one
* two
* three
*/
```

注意：如果你不使用 `indent(1)`，就不必在代码中使用 `/*-`，或为他人可能对你的代码运行 `indent(1)` 作让步。

单行注释(Single-Line Comments)

短注释可以显示在一行内，并与其后的代码具有一样的缩进层级。单行注释之前应该有一个空行。以下是一个 Java 代码中单行注释的例子：

```
if (condition) {
/* Handle the condition. */
...
}
```

尾端注释(Trailing Comments)

极短的注释可以与它们所要描述的代码位于同一行，但是应该有足够的空白来分开代码和注释。若有多个短注释出现于大段代码中，它们应该具有相同的缩进。

以下是一个 Java 代码中尾端注释的例子：

```
if (a == 2) {
    return TRUE;          /* special case */
} else {
    return isPrime(a);    /* works only for odd a */
}
```

行末注释(End-Of-Line Comments)

注释界定符 `/**`，可以注释掉整行或者一行中的一部分。它一般不用于连续多行的注释文本；然而，它可以用来注释掉连续多行的代码段。

2) 文档注释(Documentation Comments)

文档注释的格式

文档注释描述 Java 的类、接口、构造器，方法，以及字段(field)。每个文档注释都会被置于注释定界符 `/**...*/` 之中，一个注释对应一个类、接口或成员。该注释应位于声明之前，例如：

```
/**
 * The Example class provides ...
 */
public class Example { ...
```

注意：文档注释不能放在一个方法或构造器的定义块中，因为Java会将位于文档注释之后的第一个声明与其相关联。

文档标记

文档标记可以在文档注释中使用，常见的文档标记及其使用方法、作用域说明如下表：

标记	用于	目的
----	----	----

@author name	类、 接口	说明特定某一段程序代码的作者。每一个作者各有一个标记。若在 javadoc 命令行使用了“-author”标记，就会专门从生成的 HTML 文档里提取出作者信息。
@deprecated	类、 成员函数	说明该类的应用程序编程接口（API）已被废弃，因此应不再使用。
@exception name description	成员函数	说明由成员函数发出的异常。一个异常采用一个标记，并要给出异常的完整类名。
@param name description	成员函数	用来说明传递给一个成员函数的参数，其中包括参数的类型/类和用法。每个参数各有一个标记。
@return description	成员函数	若成员函数有返回值，对该返回值进行说明。应说明返回值的类型/类和可能的用途。
@since	类、成员函数	说明自从有 JDK 1.1 以来，该项已存在了多长时间。
@see ClassName	类、接口、成员 函数、字段	在文档中生成指向特定类的超文本链接。可以并且应该采用完全合法的类名。
@see ClassName#member functionName	类、接口、成员 函数、字段	在文档中生成指向特定成员函数的超文本链接。可以并且应该采用完全合法的类名。
@version text	类、接口	说明特定一段代码的版本信息。

3) javadoc工具

用于提取文档注释的工具叫作javadoc。

javadoc 只能为public（公共）和protected（受保护）成员处理注释文档。“private”（私有）和“友好”成员的注释会被忽略

2、数据类型

1) 基本数据类型

Java 语言中提供 8 种基本的数据类型，其中 6 种是数字型，一种是字符型，还有一种是布尔型。

布尔型： boolean

布尔型占一个字节，数值范围只允许 true 或 false，不可以用 0 或非 0 表示（C/C++不同）。

字符型： char

如：char achar = ‘a’ ;Java 是使用 Unicode 编码，每个字符占两个字节。

整型： byte, short, int, long

- Java 语言的整型常量的三种表示形式：

- 十进制整数：12, -654, 0。
- 八进制整数：要求以 0 开头，如：012（少用）。
- 十六进制整数：要求以 0x 开头。
- Java 中没有无符号类型

类型	占用存储空间	取值范围
byte	1 字节	-128~127
short	2 字节	$-2^{15} \sim 2^{15}-1$
int	4 字节	$-2^{31} \sim 2^{31}-1$
long	8 字节	$-2^{63} \sim 2^{63}-1$

浮点型：float, double

类型	占用存储空间	取值范围
float	4 字节	-3.403E38~3.403E38
double	8 字节	-1.798E308~1.798E308

2) 基本数据类型的转换

- boolean 类型户可以转为其他的数据类型
- 整型，字符型，浮点型的数据在混合运算中相互转换，规则如下：
 - 精度低的类型自动转换为精度高的数据类型；数据类型按精度高低排序为：byte, short, char -> int -> long -> float -> double。其中，byte, short, char 之间不会相互转换，他们三者计算时首先转换为 int 类型。
 - 精度高的数据转为精度低的数据要加上强制转换符，但可以造成精度的降级或者溢出，使用时要注意。
 - 有多种类型的数据混合运算时，系统首先自动将所有数据转换为精度最高的那一种数据类型，然后再进行计算。

5.3.5 签名

我们开发的独立应用，有多种使用方式，包括内置到系统中，或者以第三方应用安装但需要系统权限，或者以第三方应用安装不需要系统权限。前两者需要进行系统签名，而后一种则根据需要进行第三方应用签名。下面针对**系统签名**说明两种签名方式。

1) 自动签名

当应用要内置到系统中时，有两种方式，一种是以源码的方式内置，一种是以 APK 的方式内置。

以源码的方式内置在系统中时，需要编辑该应用的 Android.mk 文件。在 Android.mk 文件中，由如下命令行决定了系统签名所使用的密钥：

```
LOCAL_CERTIFICATE := XXXX
```

其中，XXXX 的选项主要包括：testkey、platform、shared、media。

如果 Android.mk 文件中不包含 LOCAL_CERTIFICATE 项配置，表明该应用不进行系统签名。

以 APK 方式内置在系统中时，如果放在指定路径例如

alps\vendor\mediatek\konkaXXXX\artifacts\out\target\product\konkaXXX\system\app，系统编译时可以对该 APK 进行自动签名，签名证书默认为 platform。

也可以新建一个文件夹，将 apk 放进去，同时配置相应的 Android.mk 文件，并在文件 alps\build\target\product\common.mk 中添加模块名使得被编译。

以“康佳通信录”为例，在工程的中新建一个目录 KonkaContacts，路径为 alps\packages\apps\KonkaContacts，文件夹内容如下图所示：

文件名	扩展名	大小	↓日期时间
..		<DIR>	13-01-16 12:02
.svn		<DIR>	12-12-20 12:42
Android.mk		0 KB	12-12-20 12:41
KonkaContacts.apk		3,654 KB	12-12-20 12:41

其中，alps\build\target\product\common.mk 的“PRODUCT_PACKAGES := ”项之后，添加如下内容：

```
# konka-chinn-KonkaContacts-20130306-start
PRODUCT_PACKAGES += KonkaContacts
```

```
# konka-chinn-KonkaContacts-20130306-end
```

需要注意的是，PRODUCT_PACKAGES 所添加的 module 名称需与模块中 Android.mk 文件里指定的 LOCAL_MODULE 保持一致。

而 Android.mk 文件内容如下：

```
LOCAL_PATH := $(call my-dir)
include $(CLEAR_VARS)
# Module name should match apk name to be installed
LOCAL_MODULE := KonkaContacts
LOCAL_MODULE_TAGS := optional
LOCAL_SRC_FILES := $(LOCAL_MODULE).apk
LOCAL_MODULE_CLASS := APPS
LOCAL_MODULE_SUFFIX := $(COMMON_ANDROID_PACKAGE_SUFFIX)
LOCAL_CERTIFICATE := PRESIGNED
include $(BUILD_PREBUILT)
```

我们看到上面的 Android.mk 文件内容中，LOCAL_CERTIFICATE 项为 PRESIGNED，表明该文件夹中的 apk 已经预先签过名，则系统编译时不会再做签名处理。而 LOCAL_CERTIFICATE 如果选择用 testkey、platform、shared、media 配置时，系统会选择相应的证书进行签名处理。

2) 手动签名

签名工具和证书在 Google 在源代码中都有提供。

签名工具 signapk 的源代码目录：build\tools\signapk，编出来是如下 Jar 执行文件：

\out\host\common\obj\JAVA_LIBRARIES\signapk_intermediates\javalib.jar

系统的签名证书目录在\build\target\product\security\project-name\，一般有四种证书：media、platform、shared、testkey。

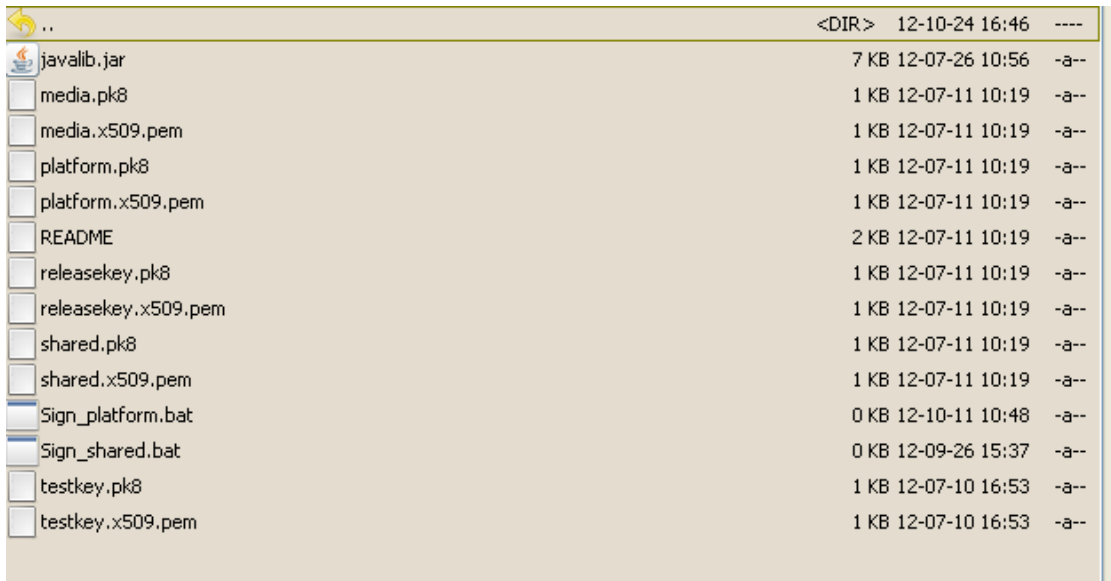
签名需执行命令，例如

```
java -jar javalib.jar shared.x509.pem shared.pk8 A.apk B.apk
```

其中，shared.x509.pem、shared.pk8 表示签名采用了 shared 系统签名证书，A.apk 为签名前的应用名称，B.apk 是签名后生成的应用名称，名称由开发者指定。

样例

手动签名可以将上述生成的工具以及各个证书放在同一个文件夹中，如下：



然后在“运行”中执行 cmd，跳到该文件夹所在目录，执行

java -jar javalib.jar XXXX.x509.pem XXXX.pk8 A.apk B.apk。

另外，也可以针对各个证书写相应的批处理文件，如上图中的 Sign_platform.bat 和 Sign_shared.bat，其内容分别如下：

Sign_platform.bat: **java -jar javalib.jar platform.x509.pem platform.pk8 A.apk.B.apk**

Sign_shared.bat: **java -jar javalib.jar shared.x509.pem shared.pk8 A.apk B.apk**

以上样例仅供参考，开发者在开发过程中进行手动签名时可以根据需要灵活处理，譬如在指定源文件目录放未签名 apk，在指目标文件目录放签名后生成的 apk，而批处理则根据情况增加文件所在目录信息。

5.3.6 版本号

对于每个独立的应用，可以在其开发包中的 AndroidManifest.xml 文件中定义版本号。例如：

```
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
package="com.konka.launcher2"
android:sharedUserId="com.konka.launcher.theme"
android:versionCode="9"
android:versionName="1.0.8.A015.S001s.O"
>
```

上述代码中，“versionCode”是个整数值，代表应用程序代码的相对版本，也就是版本更新过多少次；这部分不会显示给用户。

“versionName”是字符串值，代表应用程序的版本信息，需要显示给用户。

因此，我们根据开发版本的更新填写上述信息，其中，需要显示给用户的 versionName 需遵循以下规则：

1. 0. 0. A015_ICS. S001p. 0

			__ 市场版本标识
		__ 签名信息	
	__ SDK版本		
__ 版本编号			

补充说明:

1) 版本编号

版本编号由三位数字组成，每个数字之间由“.”做分隔，占据整个版本号的前5位（含分隔符）。版本每发放一次递增1, 如第一次发放为1.0.0, 第二次发放为1.0.1，顺次递增。

2) SDK版本

SDK版本由API level及其分支来标识，采用“A”+ “0”（数字）+ API level + “_” + “分支标识”的方式，位于整个版本号的第三和第四个分隔符之间；若无分支，则“_”+“分支标识”不需要加上。

例如Android 4.0.3所对应的API level为15，其分支标识为ICS，因此其所对应的SDK版本信息为：A015_ICS；而Android4.0.4所对应的API level也是15，其分支标识为ICS2，因此其对应的SDK版本信息为：A015_ICS2。

如果应用不针对分支，则在SDK版本信息中不需要加分支信息，譬如，应用使用API level15的SDK开发，同时适用于ICS和ICS2，则其对应的版本信息即A015。

3) 签名信息。

签名信息对应的是签名文件的版本号，其中分为系统签名和第三方应用签名。

S代表为系统签名；001则对应签名文件版本编号，该版本在签名文件中无法体现，由系统组将所需要用到的系统签名证书进行编号管理；p代表是platform证书，除此之外，还有t代表testkey证书，s代表shared证书，m代表media证书。

若应用作为第三方应用，使用第三方应用证书，签名信息项为“T”+ 编号，证书需要另行维护，并与编号对应。

4) 市场版本标识

因为应用所适应的不同市场需求，所以版本号对此进行了区分，使用“.”加大写字母作为标识。

其中，适用于内销项目版本的应用，版本号最后两位为“.D”；

适用于外销项目版本的应用，版本号最后两位为“.O”；

适用于公开电子市场的应用，版本号最后两位为“.P”。

通用版本的应用，版本号最后两位为“.A”。

5) 样例。

以前面提到的代码为例：

android:versionCode="9"

android:versionName="1.0.8.A015.S001s.O"

上述信息说明，该apk一共发布了9个版本，当前版本是1.0.8，开发所使用的API level是15，该apk需要系统签名，并且选择的是shared证书签名，该apk适用于外销机型。

六、软件测试

功能开发完成，需要开发人员自己进行基本功能测试以及交互测试。按照SPEC定义的需求，每个功能点都需要测试一遍，以保证功能的完整性；而与该功能相关的其他应用操作也需要进行测试，以保证交互的正确性。

提交测试的时候，要注明一下，哪些需要重点测试，以及相关的交互测试要点。

出现 BUG，切忌只为了规避眼前的问题而修改，应该经过冷静分析，找出BUG 出现的原因，并进行正确、不影响其他功能的修正。分析bug成因时，不应拘泥于某一个小局部，换一个想法、从另外一个角度也许让你柳暗花明。

七、代码审查

应用程序开发完成，需要对代码进行检查，可以通过代码走读及审查方式对代码进行检查。代码走读主要是对程序的编程风格如注释、命名等以及编程时易出错的内容进行检查，可由开发人员自己或开发人员交叉的方式进行；代码审查主要是对程序实现的功能及程序的稳定性、安全性、可靠性等进行检查及评审，可通过自审、交叉审核或指定部门抽查等方式进行。

八、资料整理

应用程序开发完成，需要整理下面资料，基本资料包含如下：

SPEC 设计文档、概要设计、代码、专项测试报告、移植文档（移植文档中必须包含此功能RAM 和ROM 的使用情况）

把相关信息整理好后，按各部门的要求，把全部资料提交给主管人员，进行项目结项。

九、参考资料

- 《软件编码规范 --- C&C++》 康佳研究院
- 《高质量 C++/C 编程指南》 作者：林锐
- 《C 语言软件编程规范》拟制： 刘旭光
- 匈牙利命名规范
- Java 语言编码规范(**Java Code Conventions**)